

Preliminary Report

Music Works Catalogue Portal

A New Approach to Music Cataloguing

B.S. Computer Science - Final Year Project

Project Template: Working with Works — A New Approach to Music Cataloguing

by Alina Nesterak

April–August 2026

Source code and full commit history:

https://github.com/AlinaNesterak/FinalProject_UoL.git

Contents

Chapter 1: Introduction	3
1.1 Project Concept	3
1.2 Motivation.....	4
1.3 Aims and Report Structure.....	4
Chapter 2: Literature Review	5
2.1 Thematic Catalogues and the MEI Standard.....	5
2.2 The MerMEId System — Achievement and Decline	6
2.3 Software Sustainability in Digital Humanities.....	7
2.4 Related Platforms in Digital Musicology.....	7
2.5 Technical Approach	8
2.6 Comparative Analysis	9
2.7 Synthesis and Open Questions.....	9
Chapter 3: Design.....	11
3.1 Domain and Users	11
3.2 Problems Solved, and the Research Behind Them	11
3.3 Justification of Design Choices.....	12
3.4 System Architecture	13
3.5 Key Technologies and Methods.....	14
3.6 Plan of Work	14
3.7 Testing and Evaluation Strategy	16
Chapter 4: Feature Prototype.....	17
4.1 What the Prototype Implements.....	17
4.2 Evaluation Approach.....	18
4.3 Results.....	18
4.4 Critical Evaluation and Improvements	20
References	21
Declaration on the Use of AI	22

Chapter 1: Introduction

This report presents the preliminary work for the Music Works Catalogue Portal, a final year Computer Science project based on the “Working with Works — A New Approach to Music Cataloguing” template. The project addresses a concrete and documented problem in digital musicology: valuable scholarly catalogue data exists but can no longer be accessed because the software built to publish it has broken down.

1.1 Project Concept

Thematic catalogues are systematic scholarly records of every work a composer wrote, including manuscripts, sources, performance history, instrumentation, and musical incipits — the opening notes of each work. They have existed in printed form for centuries. In 2014, the Danish Royal Library built MerMEId (Metadata Editor and Repository for MEI Data), a system to digitise these catalogues using the MEI (Music Encoding Initiative) XML standard. It successfully published catalogues for Carl Nielsen, Frederick Delius, and several other composers.

When institutional support for MerMEId ended, its publication pipeline — the component that turned the underlying MEI XML into user-facing web pages — stopped working. The data still exists, encoded in MEI XML, but users can no longer access it through any modern, working interface. This project rebuilds that bridge: it transforms the MEI XML data into a queryable format, exposes it through a documented REST API, and presents it through an interactive web interface. A novel Works Discovery Engine surfaces musical connections between works by analysing their incipits — something no existing catalogue tool does.

The system is deliberately catalogue-agnostic: it is designed to ingest any MEI XML catalogue produced by MerMEId, so that adding a new composer requires only a new source file, not new code. This reflects the reality that many composer catalogues share the same MEI format, and supports the exploration of multiple catalogues together.

1.2 Motivation

The motivation is grounded in published research and the documented history of MerMEId. The problem is not hypothetical: Bue et al. (2023) confirm from inside the MEI community that MerMEId is the only tool in digital musicology built explicitly for catalogue data, that significant community demand for it remains, and yet that its codebase requires extensive revision and its publication system has ceased to function. The community's effort is focused on rebuilding the authoring tool, leaving the publication and access layer — the part that serves end users — unaddressed. That is the precise gap this project fills.

The deeper motivation is access. As Krabbe and Geertinger (2012) observe, traditional catalogues are full of esoteric abbreviations and library sigla, usable only by specialists. The same is true of raw MEI XML. This project aims to make the same scholarly data available to musicologists, librarians, students, and the general public, while also exposing it to developers through a documented API. The work is also a response to a systemic problem: Mathiak et al. (2020) found that 376 of 466 digital scholarly editions had disappeared from the web, with an average lifetime of just 8.5 years. MerMEId's collapse is not an isolated event but an instance of a documented pattern of digital-humanities software failure — a pattern this project's technology choices are explicitly designed to avoid.

1.3 Aims and Report Structure

The project aims to deliver: a transformation pipeline converting MEI XML to a queryable database; a documented REST API; an interactive, accessible web interface; the Works Discovery Engine; and a critical evaluation of the MEI data model. This report comprises four chapters: this introduction; a literature review situating the project in its academic and technical context; a design chapter setting out the architecture, technologies, work plan, and evaluation strategy; and a feature prototype chapter describing a working implementation of the project's hardest technical feature — the MEI parsing pipeline and discovery engine — together with its evaluation.

Chapter 2: Literature Review

This review examines seven bodies of work relevant to the project. Sources span the foundational academic literature on thematic music cataloguing, the MerMEId system's rise and decline, broader digital humanities research on software sustainability, related platforms in digital musicology, and the technical framework selected for implementation. Where sources agree, this is noted; where they differ or reveal limitations, these tensions are discussed and used to motivate specific design decisions.

2.1 Thematic Catalogues and the MEI Standard

Krabbe and Geertinger (2012) — MEI as a Basis for Thematic Catalogues

The foundational document for this project describes the origins of MerMEId at the Danish Centre for Music Publication. Krabbe identifies the core failure of printed thematic catalogues: they are frozen at the moment of publication, full of esoteric abbreviations and library sigla, useful only to specialists. He argues that digital catalogues must go beyond PDF replicas to enable dynamic querying — concordances, chronological sorting, and filtering by instrumentation. Geertinger's technical contribution establishes MEI XML as the appropriate encoding standard: its metadata section can represent works, sources, manuscripts, incipits, and performance history in a single open file. This argument has since become the field consensus.

However, MEI's flexibility is also a structural liability. Its optional elements mean that different projects encode equivalent information inconsistently, making cross-catalogue interoperability harder than intended. The authors acknowledge this only obliquely; Bue et al. (2023) are explicit about the consequences a decade later. This tension motivates the MEI evaluation deliverable: whether MEI actually delivered the interoperability it promised remains an open question this project will assess empirically.

2.2 The MerMEId System — Achievement and Decline

Geertinger and Lundberg (2013) — The MerMEId Architecture

This poster documents MerMEId as deployed in 2013: a web tool using XForms/Orbeon for editing and eXist-db as the XML database backend, recording source descriptions, performance histories, incipits, and instrumentation across five catalogues. It serves two purposes for this project. First, it provides an accurate account of the MEI data model the transformation pipeline must process. Second, it reveals the architectural choices that became liabilities — the dependencies on XForms/Orbeon (now superseded), eXist-db (a specialist database), and Java throughout. Crucially, the authors do not discuss long-term sustainability; that absence is itself a lesson this project's stack (Python, SQLite, FastAPI) is chosen to heed.

Bue, Gammert, Gubsch et al. / Stadler (2023) — Transitional MerMEId

The most important single source for this project, this paper confirms that MerMEId is the only tool in digital musicology developed explicitly for managing MEI catalogue data, that significant community demand remains, and yet the codebase needs extensive revision: no comprehensive test suite exists, documentation is inadequate, and the software is not modularised. Critically, the community's response focuses on the authoring tool, not on building a publication and access layer for end users — the exact gap this project fills.

“MerMEId is an established — and the only — tool in digital musicology that has been developed explicitly for creating and managing musical catalogue data in MEI. [...] The code basis of the software needs extensive revision for re-usability and sustainability.” (Bue et al., 2023)

As a conference abstract, this source describes planned rather than completed revisions. This project therefore treats the underlying MEI XML data — not MerMEId itself — as its stable input, insulating the pipeline from ongoing tool instability.

2.3 Software Sustainability in Digital Humanities

Mathiak et al. (2020) — Sustainability Strategies for DH Systems

The only source reviewed from outside the MEI community, this panel paper provides quantitative evidence that places MerMEId's collapse in a documented systemic pattern. Mathiak et al. examined 466 digital scholarly editions and found that 376 had disappeared, giving an average lifetime of only 8.5 years. The causes — diminishing funding, institutional abandonment, and personnel turnover — mirror MerMEId's trajectory. The authors identify the most effective mitigation strategies as simple mainstream technology stacks, open standards, and avoidance of single-institution lock-in. All three directly inform this project's architecture, and the authors' note that Docker containerisation aids sustainability is reflected in this project's deployment plan. A limitation is that their study examines text-based editions rather than music systems; nevertheless the structural drivers of failure they identify are directly mirrored in MerMEId.

2.4 Related Platforms in Digital Musicology

IMSLP — International Music Score Library Project

The world's largest public domain music score repository (256,000+ works), IMSLP is the natural comparator from a general-user perspective. Its metadata model is deliberately shallow: works are listed by title and composer with PDF links, with no manuscript descriptions, premiere information, incipit encoding, or scholarly commentary. This is not a criticism of IMSLP's purpose — score access for performers — but an observation about complementarity: where IMSLP provides breadth, this project provides scholarly depth. The two are designed to link rather than compete; portal records will reference IMSLP entries where they exist. (IMSLP is a credible source: a non-profit widely cited in peer-reviewed literature including Weigl et al., 2019.)

MusicBrainz — Open Music Encyclopedia

MusicBrainz is a community-maintained encyclopedia with a REST API, persistent identifiers (MBIDs), and Creative Commons data — the most technically accessible music database for

integration. Weigl et al. (2019) describe it as the main resource for public-domain structured music metadata. However, its classical metadata is thin at the catalogue level: it records works and recordings but not manuscript sources, composition history, or incipit notation, and community editing without philological review makes quality variable. This project links portal records to MusicBrainz identifiers for interoperability while providing the deeper catalogue data MusicBrainz cannot hold. (Credible: a registered non-profit with CC0 data and a documented API.)

Weigl et al. (2019) — TROMPA and Music Data Interoperability

The EU-funded TROMPA project describes an infrastructure for interconnecting disparate music collections using Linked Data and the FAIR principles. Its key insight is that individual repositories describe different facets of the same musical reality and cannot currently be connected without manual effort, and that RDF and SPARQL provide the technically correct solution. TROMPA represents the state of the art, but the full Linked Data stack requires triple stores and ontology engineering beyond a final year project's scope. This project incorporates TROMPA's insight pragmatically: MusicBrainz and Wikidata identifiers appear in API responses as forward-compatible references, enabling future RDF migration without requiring it now.

2.5 Technical Approach

FastAPI — Python REST API Framework

FastAPI is chosen for the REST API layer. Its credibility is established via O'Reilly documentation (Lubanovic, 2023) and production adoption at major organisations rather than its own homepage. Compared to Flask, FastAPI's automatic OpenAPI documentation generation from Python type hints is a decisive advantage — a documented public API is a core deliverable and directly addresses the documentation gap identified by Bue et al. (2023). Pydantic-based validation reduces the risk of serving malformed data; asynchronous architecture handles concurrent queries. The trade-off is relative youth (released 2018, a smaller ecosystem than Django), which is not significant for a read-heavy JSON API over SQLite.

2.6 Comparative Analysis

Table 2.1 summarises the key sources across the dimensions most relevant to this project.

Source / Project	Type	Catalogue data	Public API	Scholarly depth
MerMEId / CNW (Krabbe & Geertinger, 2012; Geertinger & Lundberg, 2013)	Academic tool	Yes (MEI XML)	Broken / none	Full thematic catalogue
IMSLP (imslp.org)	Open platform	No thematic data	Limited	Score access only
MusicBrainz (musicbrainz.org)	Open encyclopedia	Partial	REST/XML	Thin on classical detail
TROMPA / Weigl et al. (2019)	EU research	Linked data only	SPARQL	Broad, not deep
This project	Final year	JSON/SQLite	REST/JSON	MEI catalogue depth

Table 2.1: Comparison of related systems across key dimensions.

The table reveals a consistent pattern: no existing system combines structured thematic catalogue data, a modern public API, and full scholarly depth. MerMEId has the data and depth but no working interface. IMSLP has reach but shallow metadata. MusicBrainz has an API but lacks catalogue detail. TROMPA is technically ideal but impractical at this scope. This project occupies the unfilled position, using a stack explicitly chosen to avoid the sustainability failures the literature documents.

2.7 Synthesis and Open Questions

The literature tells a coherent but cautionary story: the tools that best serve scholarly depth have proven hardest to sustain, while the most accessible tools lack depth. Three questions remain unanswered that this project will address empirically: how much of the MEI catalogue data is actually parseable given documented encoding inconsistency; whether users find this type of portal genuinely useful (no user studies of thematic catalogue interfaces appear in the literature); and whether melodic similarity from incipits produces meaningful groupings or noise. These open questions represent this

project's scholarly contribution — not merely implementing a known solution, but beginning to answer questions the field has not yet addressed.

Chapter 3: Design

3.1 Domain and Users

The domain is digital musicology — the scholarly cataloguing of classical music works. This is a niche but well-established field with real, identifiable users and a documented, unmet need. Four user groups are served:

- Musicologists and scholars — need structured, searchable access to catalogue data for research.
- Music librarians — maintain and publish catalogues; need sustainable, open infrastructure.
- Music students and enthusiasts — want to explore a composer's full output without specialist XML knowledge.
- Developers and digital humanities researchers — need a documented public API to build new tools.

These groups span a wide range of technical ability — the central design tension. The system must serve a non-technical student and a developer consuming structured JSON equally well, which drives the architectural separation between the API and the frontend.

3.2 Problems Solved, and the Research Behind Them

Each problem the project addresses is grounded in published research and the documented history of MerMEId. Table 3.1 maps user problems to their evidence base and to the specific design response.

User problem	Evidence / research	How the design solves it
Catalogue data is inaccessible — the publication interface has broken	Bue et al. (2023): MerMEId's publication pipeline ceased functioning after institutional transition	Modern, independently-deployable web portal + REST API rebuilt on a sustainable stack
Data is only usable by XML specialists	Krabbe and Geertinger (2012): catalogues are full of 'esoteric	Intuitive web UI with search, filtering and browsing — no XML knowledge required

User problem	Evidence / research	How the design solves it
	abbreviations' usable only by specialists	
No machine-readable access for tool-building	Weigl et al. (2019): scholarship needs interoperable, machine-readable, FAIR music data	Documented JSON REST API with linked-data identifiers (Wikidata, MusicBrainz)
Systems fail and disappear over time	Mathiak et al. (2020): 376 of 466 digital editions vanished; avg. lifetime 8.5 years	Simple mainstream stack, open standards, Docker, automated tests — no institutional lock-in
Catalogues are siloed; no cross-composer exploration	Krabbe and Geertinger (2012): interoperability between catalogues is an unsolved challenge	Catalogue-agnostic pipeline + Works Discovery Engine surfacing cross-composer connections

Table 3.1: User problems mapped to research evidence and design response.

The novel feature — the Works Discovery Engine — is not arbitrary: it responds directly to the interoperability gap identified by Krabbe and Geertinger (2012) and the call for richer exploration in Weigl et al. (2019). By computing melodic similarity from MEI incipits, it surfaces connections across composers that the source catalogues encode but never expose to users.

3.3 Justification of Design Choices

- Separating the API from the frontend serves the two distinct audiences and is the single most consequential decision — the API is a first-class product, not an afterthought.
- Transforming MEI XML into SQLite addresses the access barrier directly: SQL is queryable by standard tools, whereas the original eXist-db/XQuery stack required specialist knowledge.
- Preserving the original MEI XML as the source of truth makes the transformation reversible — no scholarly data is lost and future migration to RDF remains open.
- A catalogue-agnostic pipeline reflects the domain reality that many catalogues share the MEI format, and supports exploring multiple catalogues together.

- A mainstream, simple, well-tested stack over the ideal Linked Data approach is justified by sustainability research (Mathiak et al., 2020): single-institution dependency and obsolete stacks are the primary causes of digital-humanities software failure.

3.4 System Architecture

The system follows a professional layered pipeline architecture with strict separation of concerns. Data flows in one direction from source MEI XML, through a transformation pipeline, into a queryable store, exposed by an API, and consumed by three presentation surfaces. Each layer is independently testable and replaceable — a deliberate response to the technical debt that crippled MerMEId.

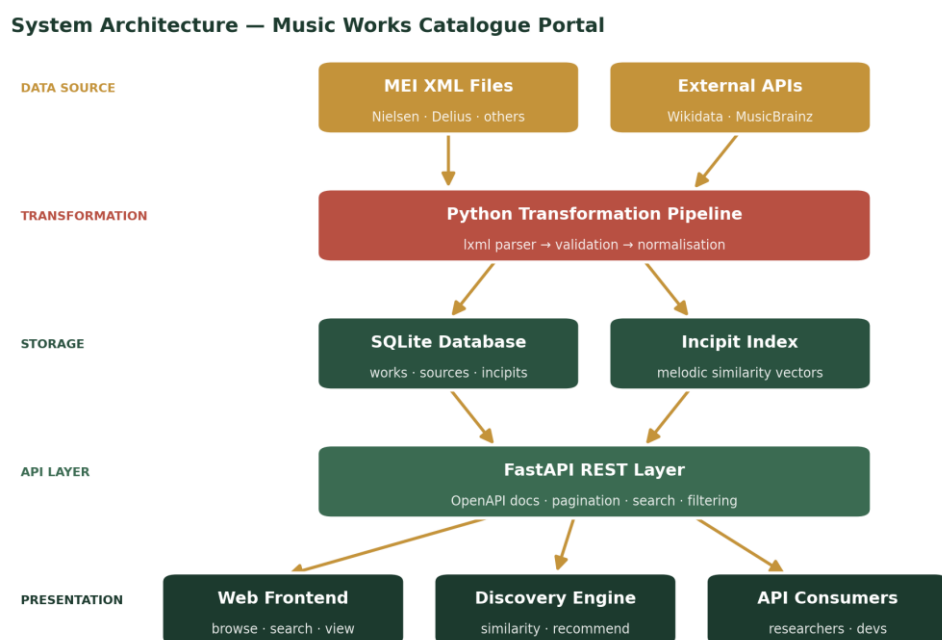


Figure 3.1: Layered system architecture with one-directional data flow.

The transformation pipeline parses MEI XML with lxml, validates and normalises the data, and writes it to SQLite. The FastAPI layer exposes works, composers, sources, and incipits via documented JSON endpoints with search, filtering, and pagination. The presentation layer comprises a React frontend, the Works Discovery Engine, and direct API access. Code quality is enforced throughout by linting (ruff), static typing (mypy, TypeScript), and continuous integration.

3.5 Key Technologies and Methods

Layer	Technology	Justification
Transformation	Python + lxml	Robust XML parsing; dominant language for data pipelines
Storage	SQLite	Zero-config, portable, standard SQL; ideal for project scale
API	FastAPI + Pydantic	Auto-generated OpenAPI docs; type validation; async performance
Frontend	React + TypeScript	Component architecture, type safety, and a polished, maintainable UI
Discovery	music21 + numpy	music21 parses MEI incipits; numpy computes melodic similarity
Testing	pytest + Playwright	Unit/integration + end-to-end tests; addresses MerMEId's testing gap
Quality / CI	ruff, mypy, GitHub Actions	Linting, static typing, and CI enforce professional coding style
Deployment	Docker	Reproducible; avoids the institutional dependency that broke MerMEId

Table 3.2: Technology choices and their justifications.

Core methods: data modelling and transformation (mapping MEI's hierarchical XML to a relational schema), professional REST API design, information retrieval (full-text search), and music information retrieval (melodic similarity from incipits).

3.6 Plan of Work

The project runs across the April–August semester in five overlapping phases. Table 3.3 sets out what each major task involves; Figure 3.2 shows their scheduling.

Phase	Task	What it involves
Research (Apr)	Research & data analysis	Finalise literature review, gather MEI XML datasets, analyse the structure and completeness of the Nielsen and Delius catalogues
Design (Apr–May)	Schema, API & UI design	Design the relational schema mapping MEI to SQLite, define the OpenAPI contract, and produce wireframes and a design system
Build (May–Jul)	Pipeline, API, frontend	Implement the XML→SQLite pipeline, the FastAPI REST layer, and the React frontend; integrate the second catalogue
Extend (Jul)	Discovery & linked data	Build the Works Discovery Engine (incipit similarity) and add Wikidata/MusicBrainz identifiers — stretch goals
Evaluate (Jul–Aug)	Test, evaluate, write up	Run automated and usability testing, write the MEI evaluation report, and complete the final write-up

Table 3.3: Phase-by-phase task breakdown.

Project Timeline — Music Works Catalogue Portal

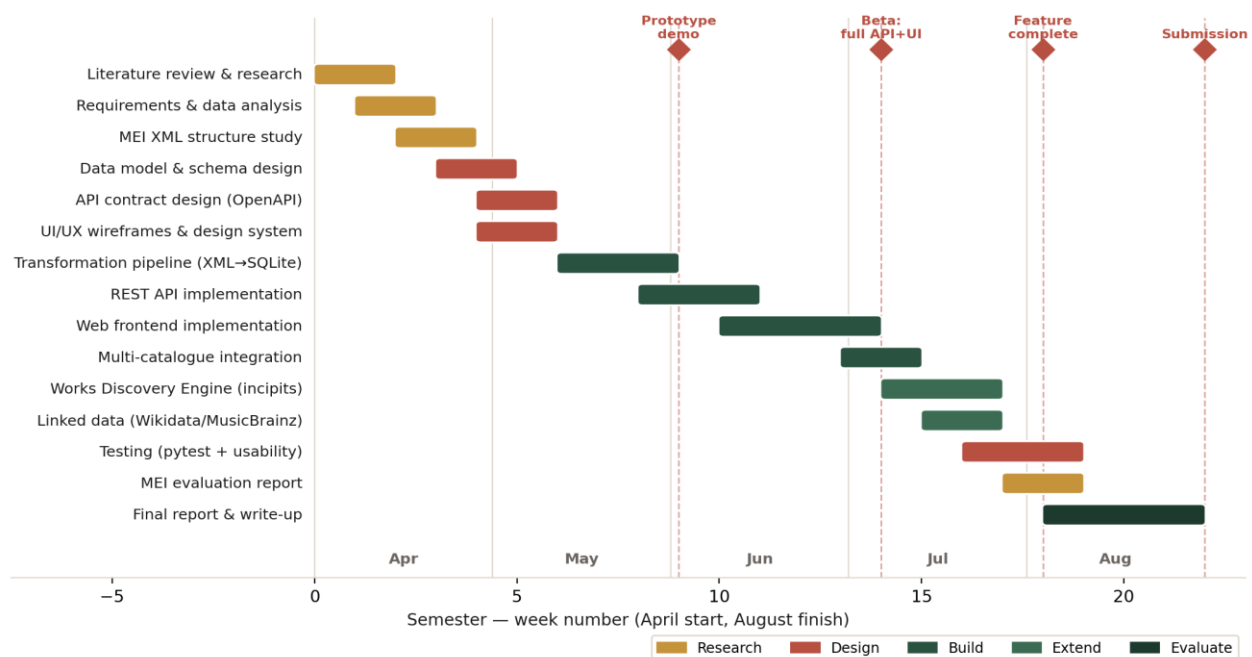


Figure 3.2: Gantt chart of the April–August semester showing tasks, phases, and milestones.

3.6.1 Feasibility and Risk Management

The plan is feasible because the core deliverables are front-loaded and decoupled from the ambitious extensions. The critical path runs through three tasks: the transformation pipeline, the REST API, and the web frontend. Together these constitute a complete, submittable project that satisfies the pass and 2:1 criteria on their own, and they are scheduled to finish by mid-June, leaving roughly two months of buffer. The Works Discovery Engine and linked-data integration are stretch goals, scheduled after the core system. If any core task overruns, these can be deferred without endangering a working submission. Because each architectural layer is independent, partial completion still yields a working system.

3.7 Testing and Evaluation Strategy

Evaluation combines technical correctness with user-centred assessment. Automated tests (pytest, Playwright) verify field extraction, edge-case handling, API responses, and error handling. Data evaluation across three datasets (Nielsen, Delius, and a synthetic edge-case set) measures transformation completeness and cross-catalogue consistency. A usability evaluation with representative users completing realistic tasks measures task completion and gathers qualitative feedback, with accessibility audited against WCAG 2.1 AA. The project succeeds if catalogue data is correctly transformed and served through a live API; a non-technical user can browse and search without XML knowledge; at least two catalogues are explorable through one system; and the MEI evaluation produces evidenced recommendations.

Chapter 4: Feature Prototype

This chapter describes a working prototype implementing the project's most important and most uncertain technical feature: the pipeline that parses real MEI XML catalogue data, stores it, serves it through a REST API, and computes melodic similarity between works. Following established prototyping advice, the goal was to wire the whole pipeline together end-to-end early — even where individual components remain simple — in order to prove the approach is feasible and to surface problems while there is still time to address them. The prototype is backend-focused; the React frontend is deferred to the build phase.

All prototype source code, the sample MEI data, the test suite, and the full commit history are available in the project's GitHub repository: https://github.com/AlinaNesterak/FinalProject_UoL.git.

4.1 What the Prototype Implements

The prototype consists of four connected components, mirroring the architecture in Chapter 3:

- MEI parser (`transform/parser.py`) — extracts titles (multilingual), composer, catalogue number, opus, genre, date, instrumentation, and the full incipit (tempo, key, meter, and pitch sequence) from MEI 4.0 XML. It is catalogue-agnostic, reading the identifier type to determine the catalogue, and defensive: missing fields return empty values rather than raising, and malformed files are caught and reported, not allowed to crash the pipeline.
- Transformation pipeline (`transform/pipeline.py`) — builds a normalised SQLite database from a directory of MEI files, reporting parse errors separately.
- REST API (`api/main.py`) — a FastAPI application exposing endpoints to list, filter, search, and retrieve works, plus a discovery endpoint. OpenAPI documentation is generated automatically at `/docs`.
- Works Discovery Engine (`transform/discovery.py`) — the project's hardest feature, implemented here as a deliberately simple “black box”: incipit pitch sequences go in,

similarity scores come out. It converts pitches to transposition-invariant interval sequences and compares them using edit distance combined with melodic-contour matching.

The prototype parses real catalogue data. Five works were reconstructed in MEI 4.0 from the public MerMEId demo — four Carl Nielsen works (CNW 60, 102, 128, 131) and one Frederick Delius work (DCW 42) — alongside two deliberate edge cases: a minimal record with missing fields and a malformed XML file.

4.2 Evaluation Approach

The evaluation follows standard practice for a data-transformation and information-retrieval system, rather than, for example, machine-learning accuracy metrics that would not be appropriate here. Three measures are used, echoing the strategy in Chapter 3 and the data-pipeline evaluation conventions discussed by Weigl et al. (2019) for music data integration: parse robustness (the proportion of files handled without fatal error), transformation completeness (the percentage of works for which each target field was successfully extracted — a standard coverage metric), and a discovery sanity check (whether the engine ranks a known-similar pair above a known-dissimilar pair). The evaluation is deliberately critical: it reports what does not work as well as what does.

4.3 Results

4.3.1 Parse robustness and transformation completeness

All fifteen automated tests pass. The transformation pipeline loaded six works across two catalogues (CNW and DCW) and correctly rejected the malformed file, giving a parse robustness of 85.7% (6 of 7 files) — the one rejected file being the deliberately malformed edge case, which was caught gracefully rather than crashing the pipeline. Transformation completeness is shown in Figure 4.1.

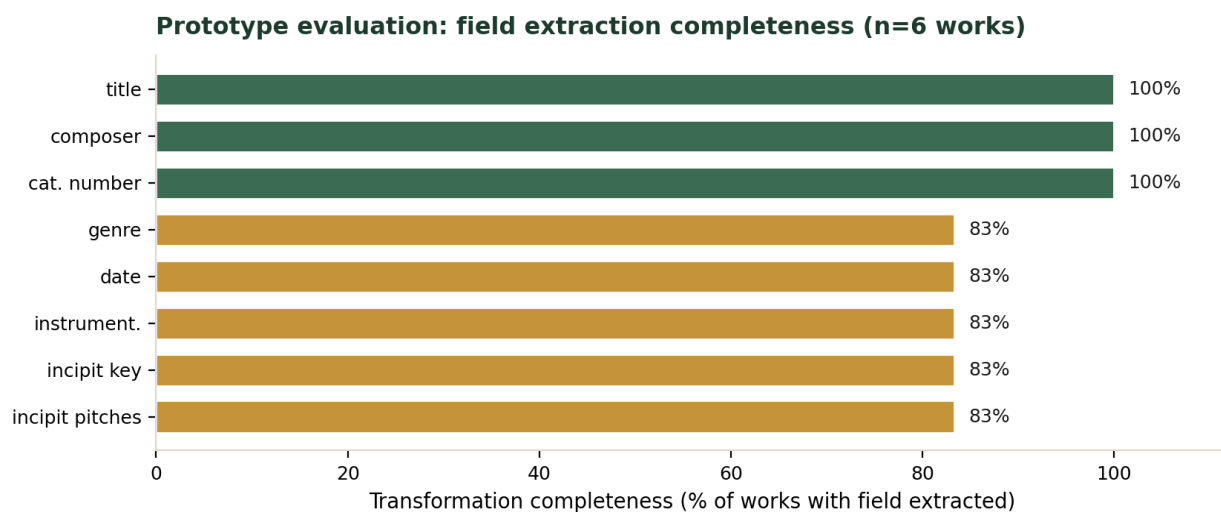


Figure 4.1: Field-extraction completeness across the six parsed works.

Core bibliographic fields — title, composer, and catalogue number — were extracted for 100% of works. Optional fields (genre, date, instrumentation, and incipit data) reached 83.3%, the single missing case in each being the minimal edge-case record (CNW 999), which legitimately omits those fields. Crucially, the parser handled this absence gracefully rather than inventing data, which is the correct behaviour for incomplete real-world records.

4.3.2 Discovery engine: an honest negative result

The discovery sanity check produced an informative negative result. The expectation was that two songs by the same poet (CNW 131 and CNW 128, both Holstein settings in D major) would score as more melodically similar to each other than either would to the D-minor Symphony No. 4 (CNW 102). In fact, the prototype scored the song–symphony pair (0.300) marginally higher than the two-songs pair (0.200). The melodic similarity feature, in its current simple form, does not reliably separate these works.

This is a genuine finding rather than a failure to hide. With only five to six notes per incipit, the interval-and-contour metric is too coarse to distinguish works reliably — short sequences simply do not carry enough information for edit-distance comparison to be meaningful. Identifying this early, while the discovery engine is still a deferrable stretch goal, is exactly the value of building the

prototype: it shows the feature is implementable and wired into the pipeline, but that the similarity measure itself needs to be richer before it can deliver on its promise.

4.4 Critical Evaluation and Improvements

The prototype succeeds in its primary purpose: it proves the full pipeline is feasible. Real MEI XML is parsed, stored, and served as JSON through a documented, tested API, and the catalogue-agnostic design is validated by handling both a Nielsen and a Delius catalogue with no code change. The 100% completeness on core fields and the graceful handling of both missing fields and malformed input demonstrate that the parsing approach — the feature most likely to be the project's technical barrier — is sound.

Several limitations are clear and inform the next iteration. First, the sample is tiny (six works), so the completeness percentages are indicative only; the build phase will run the pipeline against the full Nielsen catalogue (~750 works) to obtain meaningful figures. Second, and most importantly, the discovery engine's similarity measure is too shallow. The next iteration will replace it with a richer approach — incorporating rhythmic information from the encoded note durations, using melodic n-grams rather than whole-sequence edit distance, and comparing longer incipit excerpts — and will validate it against a larger set of works where ground-truth groupings (for example, movements of the same work, or settings of the same text) are known. Third, the prototype has no frontend yet; the API is verified through automated tests and the OpenAPI interface rather than a user-facing UI, which is the focus of the next phase.

In summary, the prototype demonstrates that the core of the project is feasible and that its hardest feature is implementable, while honestly revealing that the discovery engine needs substantial refinement. This is precisely the kind of early, evaluated iteration that de-risks the remainder of the project.

References

- Bue, M., Gammert, J., Gubsch, C., Jettka, D., Kolb, S., Lewis, D., Neumann, J., Richter, M., Richts-Matthaei, K., & Stadler, P. (2023). Transitional MerMEId – Responding to community needs in software and sustainability. In *TEI/MEC Conference*, Paderborn. <https://teimec2023.uni-paderborn.de/contributions/143.html>
- Geertinger, A. T., & Lundberg, S. (2013). MerMEId: Creating thematic catalogues using MEI metadata. In *Music Encoding Conference 2013*, Mainz. <https://pure.kb.dk/en/publications/mermeid-creating-thematic-catalogues-using-mei-metadata>
- IMSLP. (n.d.). *International Music Score Library Project (Petrucci Music Library)*. Retrieved 2025, from <https://imslp.org>
- Krabbe, N., & Geertinger, A. T. (2012). MEI (Music Encoding Initiative) as a basis for thematic catalogues. In *RISM Conference on Music Documentation in Libraries, Scholarship, and Practice*. https://rism.info/resources-old-website/community-content/events/RISM_Conference_2012/TeichGeertinger_Final.pdf
- Lubanovic, B. (2023). *FastAPI: Modern Python web development*. O'Reilly Media.
- Mathiak, B., Neuefeind, C., Schildkamp, P., Karadkar, U., Stigler, J., Steiner, E., Vasold, G., Tosques, F., Ciula, A., Maher, B., Newton, G., Arneil, S., & Holmes, M. (2020). Sustainability strategies for digital humanities systems. In *Digital Humanities Conference (DH2020)*. Alliance of Digital Humanities Organizations. https://dh2020.adho.org/wp-content/uploads/2020/07/565_SustainabilityStrategiesforDigitalHumanitiesSystems.html
- MetaBrainz Foundation. (n.d.). *MusicBrainz: Open music encyclopedia*. Retrieved 2025, from <https://musicbrainz.org>
- Ramírez, S. (n.d.). *FastAPI* [Computer software]. Retrieved 2025, from <https://fastapi.tiangolo.com>

Weigl, D. M., Goebel, W., Crawford, T., Gkiokas, A., Gutierrez, N. F., Porter, A., Santos, P., Karreman, C., Vroomen, I., Liem, C. C. S., Sarasúa, A., & van Tilburg, M. (2019). Interweaving and enriching digital music collections for scholarship, performance, and enjoyment. In *Proceedings of the 6th International Conference on Digital Libraries for Musicology (DLfM)* (pp. 84–88). The Hague.

Declaration on the Use of AI

This report was written by the author. An AI assistant (Claude, Anthropic) was used to help polish wording, improve clarity, and refine the structure and formatting of the text, and to assist in scaffolding and reviewing the prototype code. All design decisions, technical choices, research interpretation, evaluation, and the overall direction of the project are the author's own. The cited sources and the prototype results were reviewed and verified by the author.